

Container-Klassen

01 | Listen

Container-Klasse	Zentrale Methoden	Komplexität (durchschnittlich)
ArrayList	<code>add(E e)</code> , <code>get(int index)</code> , <code>remove(int index)</code> , <code>set(int index, E element)</code> , <code>size()</code> , <code>clear()</code> , <code>contains(Object o)</code>	Hinzufügen: $O(1)$. Zugriff: $O(1)$. Entfernen: $O(n)$
LinkedList	<code>add(E e)</code> , <code>get(int index)</code> , <code>remove(int index)</code> , <code>set(int index, E element)</code> , <code>size()</code> , <code>clear()</code> , <code>contains(Object o)</code>	Hinzufügen: $O(1)$. Zugriff: $O(n)$. Entfernen: $O(1)$

Methode	Beschreibung
<code>add(E e)</code>	Fügt ein Element am Ende der Liste hinzu. Bei ArrayList wird das interne Array bei Bedarf neu dimensioniert. Bei LinkedList wird ein neuer Knoten erstellt und hinzugefügt.
<code>get(int index)</code>	Greift auf das Element an einem bestimmten Index zu. Bei ArrayList erfolgt der Zugriff direkt über den Array-Index ($O(1)$). Bei LinkedList wird die Liste durchlaufen, um den Knoten zu finden ($O(n)$).
<code>remove(int index)</code>	Entfernt das Element an einem bestimmten Index. Bei ArrayList werden alle nachfolgenden Elemente verschoben. Bei LinkedList werden Knoten neu verknüpft und die Liste durchlaufen, wenn nötig.
<code>set(int index, E element)</code>	Ersetzt das Element an einem bestimmten Index durch das neue Element. Bei ArrayList erfolgt der Zugriff direkt. Bei LinkedList wird die Liste bis zum Index durchlaufen und der Knoten ersetzt.
<code>size()</code>	Gibt die Anzahl der Elemente zurück. Bei ArrayList wird eine interne Zählvariable verwendet. Bei LinkedList wird durch die Liste iteriert, um die Größe zu berechnen (sofern nicht intern gespeichert).
<code>clear()</code>	Entfernt alle Elemente aus der Liste. Bei ArrayList wird das Array auf <code>null</code> gesetzt. Bei LinkedList werden alle Knoten entfernt und die Verknüpfungen auf <code>null</code> gesetzt.
<code>contains(Object o)</code>	Überprüft, ob ein Element in der Liste vorhanden ist. Bei ArrayList wird die Liste durchlaufen und die Elemente verglichen. Bei LinkedList wird die Liste ebenfalls durchlaufen und die Knoten überprüft.

02 | Sets

Container-Klasse	Zentrale Methoden	Komplexität (durchschnittlich)
EnumSet	<code>add(E e)</code> , <code>remove(Object o)</code> , <code>contains(Object o)</code> , <code>size()</code> , <code>clear()</code>	Hinzufügen: $O(1)$. Zugriff: $O(1)$. Entfernen: $O(1)$
HashSet	<code>add(E e)</code> , <code>remove(Object o)</code> , <code>contains(Object o)</code> , <code>size()</code> , <code>clear()</code>	Hinzufügen: $O(1)$. Zugriff: $O(1)$. Entfernen: $O(1)$
TreeSet	<code>add(E e)</code> , <code>remove(Object o)</code> , <code>contains(Object o)</code> , <code>size()</code> , <code>clear()</code> , <code>first()</code> , <code>last()</code>	Hinzufügen: $O(\log n)$. Zugriff: $O(\log n)$. Entfernen: $O(\log n)$

Set-Typ	Methode	Beschreibung
EnumSet , HashSet , TreeSet	<code>add(E e)</code>	Fügt ein Element zum Set hinzu. Bei EnumSet erfolgt dies mit $O(1)$ durch Bitmasken. Bei HashSet erfolgt es mit $O(1)$, möglicherweise durch Rehashing, und bei TreeSet mit $O(\log n)$, da die Elemente sortiert eingefügt werden.
	<code>remove(Object o)</code>	Entfernt ein Element aus dem Set. Bei EnumSet erfolgt dies mit $O(1)$ durch Bitmasken. Bei HashSet mit $O(1)$, basierend auf dem Hashwert, und bei TreeSet mit $O(\log n)$, da die Struktur neu angepasst werden muss.
	<code>contains(Object o)</code>	Überprüft, ob ein Element im Set vorhanden ist. Bei EnumSet erfolgt dies mit $O(1)$, bei HashSet mit $O(1)$, basierend auf dem Hashwert, und bei TreeSet mit $O(\log n)$, da die Suche durch den Baum erfolgt.
	<code>size()</code>	Gibt die Anzahl der Elemente im Set zurück. Bei allen Sets erfolgt dies mit $O(1)$, da die Anzahl intern gespeichert wird.
	<code>clear()</code>	Entfernt alle Elemente aus dem Set. Bei EnumSet und HashSet erfolgt dies mit $O(1)$, während es bei TreeSet mit $O(n)$ erfolgt, da jeder Knoten im Baum gelöscht werden muss.
TreeSet	<code>first()</code>	Gibt das kleinste Element des Sets zurück. $O(\log n)$, da die Struktur durchlaufen wird, um das erste Element zu finden.
	<code>last()</code>	Gibt das größte Element des Sets zurück. $O(\log n)$, da die Struktur durchlaufen wird, um das letzte Element zu finden.

03 | Maps

Container-Klasse	Zentrale Methoden	Komplexität (durchschnittlich)
HashMap	<code>put(K key, V value)</code> , <code>get(Object key)</code> , <code>remove(Object key)</code> , <code>containsKey(Object key)</code> , <code>containsValue(Object value)</code>	Einfügen: $O(1)$. Zugriff: $O(1)$. Entfernen: $O(1)$
LinkedHashMap	<code>put(K key, V value)</code> , <code>get(Object key)</code> , <code>remove(Object key)</code> , <code>containsKey(Object key)</code> , <code>containsValue(Object value)</code>	Einfügen: $O(1)$. Zugriff: $O(1)$. Entfernen: $O(1)$
TreeMap	<code>put(K key, V value)</code> , <code>get(Object key)</code> , <code>remove(Object key)</code> , <code>containsKey(Object key)</code> , <code>firstKey()</code> , <code>lastKey()</code>	Einfügen: $O(\log n)$. Zugriff: $O(\log n)$. Entfernen: $O(\log n)$

Map-Typ	Methode	Beschreibung
HashMap, LinkedHashMap	<code>put(K key, V value)</code>	Fügt ein Schlüssel-Wert-Paar in die Map ein. Bei HashMap und LinkedHashMap erfolgt dies mit $O(1)$, wobei der Hashwert des Schlüssels verwendet wird. Bei LinkedHashMap wird die Einfügereihenfolge zusätzlich beibehalten.
	<code>get(Object key)</code>	Gibt den Wert für den angegebenen Schlüssel zurück. Bei HashMap und LinkedHashMap erfolgt der Zugriff mit $O(1)$, basierend auf dem Hashwert des Schlüssels.
	<code>remove(Object key)</code>	Entfernt das Schlüssel-Wert-Paar aus der Map. $O(1)$ für HashMap und LinkedHashMap , da der Hashwert des Schlüssels verwendet wird.
	<code>containsKey(Object key)</code>	Überprüft, ob der Schlüssel in der Map vorhanden ist. $O(1)$ für HashMap und LinkedHashMap .
	<code>containsValue(Object value)</code>	Überprüft, ob der Wert in der Map vorhanden ist. $O(n)$ für beide, da die Werte durchlaufen werden müssen.
TreeMap	<code>put(K key, V value)</code>	Fügt ein Schlüssel-Wert-Paar in die Map ein. $O(\log n)$, da die Map in einer sortierten Baumstruktur organisiert ist.
	<code>get(Object key)</code>	Gibt den Wert für den angegebenen Schlüssel zurück. $O(\log n)$, da die Map eine geordnete Baumstruktur verwendet.

Map-Typ	Methode	Beschreibung
	<code>remove(Object key)</code>	Entfernt das Schlüssel-Wert-Paar aus der Map. $O(\log n)$, da der Baum angepasst werden muss.
	<code>containsKey(Object key)</code>	Überprüft, ob der Schlüssel in der Map vorhanden ist. $O(\log n)$, da die Map eine geordnete Baumstruktur verwendet.
	<code>firstKey()</code>	Gibt den kleinsten Schlüssel der Map zurück. $O(\log n)$, da der erste Knoten im Baum gesucht wird.
	<code>lastKey()</code>	Gibt den größten Schlüssel der Map zurück. $O(\log n)$, da der letzte Knoten im Baum gesucht wird.

04 | Warteschlangen

Container-Klasse	Zentrale Methoden	Komplexität (durchschnittlich)
ArrayDeque	<code>add(E e)</code> , <code>remove()</code> , <code>peek()</code> , <code>offer(E e)</code> , <code>poll()</code> , <code>size()</code> , <code>clear()</code>	Hinzufügen: $O(1)$. Zugriff: $O(1)$. Entfernen: $O(1)$
PriorityQueue	<code>offer(E e)</code> , <code>poll()</code> , <code>peek()</code> , <code>size()</code> , <code>clear()</code> , <code>contains(Object o)</code>	Einfügen: $O(\log n)$. Zugriff: $O(1)$. Entfernen: $O(\log n)$

Queue-Typ	Methode	Beschreibung
ArrayDeque, PriorityQueue	<code>add(E e)</code>	Fügt ein Element in die Queue ein. ArrayDeque: Hinzufügen am Ende. PriorityQueue: Hinzufügen entsprechend der Priorität. ArrayDeque: $O(1)$. PriorityQueue: $O(\log n)$.
	<code>remove()</code>	Entfernt und gibt das erste Element der Queue zurück. ArrayDeque: $O(1)$, bei PriorityQueue: $O(\log n)$, da die Queue nach der Entfernung reorganisiert werden muss.
	<code>peek()</code>	Gibt das erste Element der Queue zurück, ohne es zu entfernen. ArrayDeque: $O(1)$. PriorityQueue: Gibt das Element mit der höchsten Priorität zurück, $O(1)$.
	<code>offer(E e)</code>	Fügt ein Element in die Queue ein. ArrayDeque: Hinzufügen am Ende. PriorityQueue: Hinzufügen entsprechend der Priorität. ArrayDeque: $O(1)$. PriorityQueue: $O(\log n)$.

Queue-Typ	Methode	Beschreibung
	<code>poll()</code>	Entfernt und gibt das erste Element der Queue zurück. ArrayDeque: $O(1)$, PriorityQueue: $O(\log n)$, da das Element mit der höchsten Priorität entfernt wird und die Queue neu organisiert wird.
	<code>size()</code>	Gibt die Anzahl der Elemente in der Queue zurück. $O(1)$.
	<code>clear()</code>	Entfernt alle Elemente aus der Queue. $O(n)$ bei beiden, da die interne Datenstruktur durchlaufen und gelöscht werden muss.
PriorityQueue	<code>contains(Object o)</code>	Überprüft, ob ein Element in der Queue vorhanden ist. $O(n)$, da alle Elemente überprüft werden müssen.